UNIVERSIDAD AUTÓNOMA GABRIEL RENÉ MORENO

SCHOOL OF ENGINEERING — UNIDAD DE POSGRADO

Facultad de Ingeniería en Ciencias de la Computación y Telecomunicaciones

Diplomado: Cloud Infrastructure & DevSecOps

Módulo: Arquitectura de Nube y Redes Empresariales

Actividad: Laboratorio 3 — Seguridad Perimetral: Security Groups, NACLs y Gateways

Docente: Msc. Diego Alejandro Rivera Montaño

Estudiante: Elmer Fuentes Lozano

Fecha: 11/10/2026

SANTA CRUZ — BOLIVIA

Propósito y alcance

Este laboratorio continúa el diseño realizado en el Laboratorio 2 para una empresa de comercio electrónico. Tomo como base la VPC 10.0.0.0/16 y sus seis subredes en dos zonas de disponibilidad. En esta etapa defino políticas de mínimo privilegio, comparo los dos niveles de filtrado de AWS, documento la salida controlada a Internet y establezco un punto de administración de los recursos internos.

Se desarrolla exclusivamente la Ruta B: investigación equivalente a la implementación en consola. No se creó ni modificó infraestructura AWS ni se realizaron conexiones reales. Los comandos y pruebas que presento son procedimientos reproducibles y resultados esperados, no ejecuciones comprobadas. El enfoque de NAT en esta ruta es el NAT Gateway administrado, mientras que la Ruta A de la guía utiliza una NAT Instance.

Organización del informe

1. Fundamentos conceptuales (siete preguntas)
2. Matriz de reglas de mínimo privilegio
3. Arquitectura de seguridad y rutas
4. Manual de configuración en AWS
5. Comandos AWS CLI y explicación de parámetros
6. Verificaciones, errores y reflexión
7. Evidencia documental pendiente y revisión de rúbrica
8. Referencias oficiales

Continuidad: inventario heredado del Laboratorio 2

Capa / AZ	Subred del diseño	CIDR	Rol
Web A	sn-web-zona-a	10.0.0.0/23	Pública
Web B	sn-web-zona-b	10.0.2.0/23	Pública
Aplicación A	sn-app-zona-a	10.0.4.0/24	Privada
Aplicación B	sn-app-zona-b	10.0.5.0/24	Privada
Datos A	sn-datos-zona-a	10.0.6.0/26	Privada
Datos B	sn-datos-zona-b	10.0.6.64/26	Privada

En la arquitectura base se mantienen rt-publica-ecommerce (10.0.0.0/16 → local y 0.0.0.0/0 → Internet Gateway) y rt-privada-ecommerce (solo 10.0.0.0/16 → local). La salida por NAT estudiada aquí constituye una ampliación futura de la segunda tabla, sin reemplazar el direccionamiento del proyecto.

I. Fundamentos conceptuales

1. Principio de mínimo privilegio aplicado a las capas web, aplicación y datos

Entiendo el principio de mínimo privilegio como permitir únicamente las comunicaciones necesarias para realizar una tarea. En una tienda virtual esto significa que el cliente externo puede conectarse al servidor web por HTTPS, pero no debería abrir directamente una sesión con el motor de base de datos. La capa web pide información a la aplicación; la aplicación consulta datos; y el acceso de mantenimiento se canaliza por un bastión controlado. Cada servicio mantiene su función sin recibir accesos que no necesita. [1, 2]

Para este diseño permitiría TCP 443 (y TCP 80 solo para redirección a HTTPS) desde Internet hacia web. El puerto 8080 se limitaría a tráfico originado por las instancias con sg-web, y PostgreSQL TCP 5432 solo desde sg-app. SSH en las capas privadas lo aceptaría únicamente desde sg-bastion. En sentido de salida tampoco dejaría Allow all por costumbre: cada grupo autorizaría sus servicios de destino específicos. La matriz del apartado II desarrolla estos flujos con mayor precisión.

Usar 0.0.0.0/0 como origen de una regla de entrada privada significaría permitir candidatos de conexión desde cualquier IPv4. Aunque otras barreras pudieran bloquearlos, no hay justificación de negocio para ese alcance en una base de datos o servidor de lógica interna. Por eso la regla de entrada para datos debería referenciar sg-app, y no el mundo exterior. [2]

2. Diferencias y complementariedad entre Security Groups y Network ACLs

Los Security Groups actúan sobre las interfaces de red asociadas a recursos de AWS. Son controles con estado: al aceptar una conexión, el tráfico de respuesta se permite automáticamente. Sus reglas son únicamente de permiso (allow), no de rechazo explícito. Por ejemplo, si la aplicación permite una conexión desde la capa web por TCP 8080, la respuesta no requiere habilitar manualmente todos los puertos efímeros en la salida del Security Group. [1, 2]

Una Network ACL, en cambio, se asocia a una o más subredes y filtra tanto entrada como salida, con reglas numeradas de menor a mayor que pueden permitir o denegar. No recuerda conexiones anteriores. Por eso, si una NACL permite la entrada de una solicitud, también debe permitir su respuesta en la dirección contraria y el puerto correspondiente. No basta con copiar las reglas de un Security Group a una NACL. [3, 4]

Criterio	Security Group	Network ACL
Alcance	Recurso / interfaz de red (ENI)	Subred completa
Estado	Con estado (stateful)	Sin estado (stateless)
Reglas	Solo permitir	Permitir y denegar
Evaluación	Conjunto de permisos aplicables	Primera coincidencia por número ascendente

Criterio	Security Group	Network ACL
Retorno	Se autoriza para conexiones permitidas	Debe autorizarse explícitamente
Referencias	Puede usar otro SG como origen/destino	Utiliza CIDR de direcciones, no referencias SG

En una arquitectura segura utilizaría ambos controles: los SG expresan relaciones entre servicios, mientras las NACL sirven de límite adicional para la red completa. Ninguno sustituye al otro. Un error en una regla no debería bastar para exponer la base de datos a Internet.

3. Matriz completa de comunicaciones de mínimo privilegio

La propuesta detallada está en la sección II. Para construirla asumí una aplicación web que escucha en TCP 8080, una base PostgreSQL en TCP 5432 y servidores Linux administrados por SSH en TCP 22. Son elecciones del caso de estudio; en una implementación se ajustarían a los puertos reales. Además se consideran actualizaciones salientes HTTPS y el tráfico de retorno.

Separé reglas de acceso entrante de las reglas de salida iniciada. Para los SG usé referencias entre grupos y una IP de administrador de tipo /32. Para las NACL usé CIDR, porque allí no se pueden definir reglas que referencien sg-web o sg-app. El símbolo [MI_IP]/32 representa un dato por completar, no una IP inventada.

4. Función de Internet Gateway frente a NAT Gateway

El Internet Gateway conecta una VPC con Internet y puede ser destino de una ruta por defecto para subredes públicas. Por sí solo no convierte cada instancia en accesible: el recurso también necesita direccionamiento y reglas compatibles. En el Lab 2 únicamente las dos subredes web tenían 0.0.0.0/0 hacia el IGW. [5]

Un NAT Gateway público permite que recursos privados inicien conexiones hacia Internet sin aceptar conexiones entrantes no solicitadas desde ese exterior. Se coloca normalmente en una subred pública y dispone de una Elastic IP; la tabla privada apunta 0.0.0.0/0 al NAT Gateway, y la tabla pública continúa apuntando al IGW. No se crea una ruta privada directa al IGW porque eso cambiaría el modelo de exposición definido para aplicación y datos. [6, 7]

En este laboratorio justificaría NAT para la capa de aplicación si debe descargar actualizaciones o consultar un servicio HTTPS externo. Para la capa de datos mantendría la política más restrictiva, sin salida general salvo que aparezca una necesidad explícita. Por simplicidad de la guía, una tabla privada compartida con ruta NAT también daría capacidad de salida a datos; documentaré esa diferencia y una mejora de tablas por capa antes de considerar el diseño productivo.

5. Por qué utilizar un bastión en lugar de SSH directo a cada instancia

El bastión o jump host sirve como punto de entrada administrativo único. En vez de dar IP pública y abrir SSH desde Internet a cada servidor privado, solo el bastión recibiría conexiones de administración desde mi dirección pública actual (/32). Una vez dentro, el administrador alcanzaría por IP privada a las instancias de aplicación o datos según su autorización. [8]

En el caso de la tienda en línea, esto reduce la cantidad de sistemas expuestos y facilita concentrar controles de acceso, registros y revisión de intentos. El bastión no reemplaza los Security Groups de cada servidor: esos grupos seguirán autorizando TCP 22 únicamente desde sg-bastion, nunca desde cualquier IPv4. Además, la instancia bastión debe mantenerse actualizada y preferiblemente utilizar autenticación robusta o una alternativa como Session Manager, que AWS documenta para reducir los puertos públicos. [8]

6. NAT Gateway administrado frente a NAT Instance autoadministrada

Aspecto	NAT Gateway administrado	NAT Instance en EC2
Administración	AWS opera el servicio de traducción	Se administra SO, paquetes y reglas NAT
Disponibilidad	Redundancia dentro de AZ para NAT zonal	Un único equipo es punto de falla salvo replicación
Escala	Escala de forma administrada	Limitada por tamaño y red de la instancia
Costo	Cargo por hora y GB procesado, más EIP/tráfico aplicable	Horas EC2, EBS, IPv4 pública y transferencia según uso
Seguridad	No se le adjunta un Security Group	Su interfaz sí puede usar SG
Caso docente	Ruta B: investigar servicio productivo	Ruta A: alternativa de bajo costo para práctica

La Ruta A usa NAT Instance para evitar cargos propios del NAT Gateway, pero requiere deshabilitar el source/destination check, habilitar IP forwarding y configurar traducción con iptables u otra herramienta. En cambio, el NAT Gateway reduce administración y ofrece mayor capacidad. En un servicio comercial con continuidad operativa escogería un diseño NAT administrado por AZ y tablas privadas coherentes con cada zona; en una práctica controlada puede bastar con documentarlo sin crearlo. [6, 9]

Nota técnica. AWS también documenta NAT Gateway regional en versiones recientes. Para mantener la continuidad del caso y explicar las AZ, este trabajo desarrolla explícitamente el NAT Gateway público zonal; no lo presenta como la única modalidad que existe. [10]

7. Diseño que habría prevenido accesos no autorizados a la base de datos

En el escenario del proyecto, la combinación clave sería separar responsabilidades. Las subredes web seguirían siendo el único punto público mediante IGW, y su SG aceptaría únicamente HTTP/HTTPS desde Internet. Las instancias de aplicación quedarían sin IP pública y recibirían TCP 8080 desde sg-web; las instancias de datos aceptarían exclusivamente PostgreSQL TCP 5432 desde sg-app. El bastión concentraría SSH y lo restringiría a la IP autorizada del administrador.

Añadiría NACL independientes por capa para bloquear tráfico no previsto y permitir los retornos necesarios. El NAT Gateway público, si se habilita para actualizaciones, se utilizaría solo como salida iniciada desde privadas; nunca abriría una entrada pública directa a la base de datos. La protección no dependería de un solo ajuste: rutas privadas, Security Groups por servicio, NACL por subred, ausencia de IP públicas y acceso administrativo limitado trabajan como barreras complementarias. Para comprobar el diseño no bastaría con ver nombres de recursos: revisaría asociaciones, reglas efectivas y pruebas autorizadas. [1, 3, 6]

II. Matriz de reglas de mínimo privilegio

2.1 Supuestos y alcance del caso

El direccionamiento permanece idéntico al Lab 2. Para expresar políticas por grupo, denomino sg-web, sg-app, sg-datos y sg-bastion a las etiquetas funcionales de los Security Groups. En la API, el nombre real del grupo será ecommerce-web, ecommerce-app, ecommerce-datos y ecommerce-bastion, porque la documentación actual de AWS indica que un GroupName no puede comenzar por sg-. La etiqueta Name sí puede mostrar sg-web. [1, 11]

Conjunto	CIDR agregado válido	Componentes
Web A/B	10.0.0.0/22	10.0.0.0/23 y 10.0.2.0/23
App A/B	10.0.4.0/23	10.0.4.0/24 y 10.0.5.0/24
Datos A/B	10.0.6.0/25	10.0.6.0/26 y 10.0.6.64/26

Nota técnica. Los CIDR agregados se utilizan para simplificar las NACL, no sustituyen los CIDR de las seis subredes. Una referencia de Security Group se usa en SG, no en NACL.

2.2 Matriz de flujos del negocio

Origen	Destino	Prot.	Puerto	Dirección	Justificación
Internet	Capa web	TCP	443	Entrada a web	Servicio HTTPS a clientes
Internet	Capa web	TCP	80	Entrada a web	Redirección opcional a HTTPS
Web / sg-web	Aplicación / sg-app	TCP	8080	Web sale; app entra	Solicitudes a la lógica de negocio
App / sg-app	Datos / sg-datos	TCP	5432	App sale; datos entra	Consulta PostgreSQL
Mi IP pública /32	Bastión / sg-bastion	TCP	22	Entrada al bastión	Administración autorizada
Bastión / sg-bastion	Capa web	TCP	22	Bastión sale; web entra	Mantenimiento web
Bastión / sg-bastion	Capa app	TCP	22	Bastión sale; app entra	Mantenimiento privado
Bastión / sg-bastion	Capa datos	TCP	22	Bastión sale; datos entra	Mantenimiento controlado de datos

Origen	Destino	Prot.	Puerto	Dirección	Justificación
App / sg-app	Servicio HTTPS externo	TCP	443	Salida vía NAT	Actualizaciones o API autorizada
Web / sg-web	Servicio HTTPS externo	TCP	443	Salida hacia IGW	Actualización del host o dependencias
Datos	Internet	—	—	Denegar (ausencia de regla)	No se requiere acceso externo directo

Estas son las conexiones que se iniciarían. Los paquetes de respuesta de una conexión aceptada son administrados automáticamente por Security Groups, pero deben tener reglas de retorno en las NACL cuando cruzan límites de subred. DNS, servicios de gestión y puertos diferentes deberán justificarse y agregarse solo si la aplicación concreta los utiliza.

2.3 Reglas propuestas para Security Groups (entrada/salida)

Etiqueta SG	Sentido	Servicio	Origen / destino	Finalidad
sg-web	IN	TCP 80/443	0.0.0.0/0	Publicación de la web; considerar HTTPS como principal
sg-web	IN	TCP 22	sg-bastion	Administración desde el bastión
sg-web	OUT	TCP 8080	sg-app	Petición a aplicación
sg-web	OUT	TCP 443	0.0.0.0/0	Actualizaciones autorizadas
sg-app	IN	TCP 8080	sg-web	Única entrada funcional desde web
sg-app	IN	TCP 22	sg-bastion	Mantenimiento privado
sg-app	OUT	TCP 5432	sg-datos	Consulta a PostgreSQL

Etiqueta SG	Sentido	Servicio	Origen / destino	Finalidad
sg-app	OUT	TCP 443	0.0.0.0/0	Conexión iniciada a través de NAT
sg-datos	IN	TCP 5432	sg-app	Consultas de aplicación
sg-datos	IN	TCP 22	sg-bastion	Mantenimiento controlado
sg-datos	OUT	—	—	Sin nuevas conexiones salientes por defecto
sg-bastion	IN	TCP 22	[MI_IP]/32	Solo equipo administrador
sg-bastion	OUT	TCP 22	sg-web, sg-app, sg-datos	Administración de servidores
sg-bastion	OUT	TCP 443	0.0.0.0/0	Actualización del host bajo control

Nota técnica. En grupos nuevos AWS autoriza de forma predeterminada toda la salida IPv4. El procedimiento incluye revocar esa regla antes de configurar las salidas limitadas, ya que agregar reglas restrictivas no anula un Allow all que aún existe. Las NACL no usan IDs de SG como origen. [2, 11]

2.4 Ejemplo detallado de reglas de NACL con tráfico de retorno

Para evitar confundir “servicio de destino” con “puerto de respuesta”, la tabla separa entrada y salida. En una NACL el puerto indicado es el puerto de destino del paquete que cruza el límite de la subred. Se usa 1024–65535 en retornos por compatibilidad con clientes heterogéneos; en producción se podría ajustar con conocimiento del sistema. La primera coincidencia por orden decide. Todas las reglas ausentes terminan en DENY *. [3, 4]

NACL	Flujo	N.º	Puerto/protocolo	CIDR origen/destino	Motivo
nacl-web	IN	100	TCP 80	0.0.0.0/0	HTTP para redirección
nacl-web	IN	105	TCP 443	0.0.0.0/0	HTTPS público

NACL	Flujo	N.º	Puerto/protocolo	CIDR origen/destino	Motivo
nacl-web	IN	110	TCP 22	[MI_IP]/32	SSH al bastión
nacl-web	IN	120	TCP 1024-65535	0.0.0.0/0	Retorno de conexiones salientes del bastión y NAT
nacl-web	OUT	100	TCP 1024-65535	0.0.0.0/0	Respuestas de web y SSH
nacl-web	OUT	110	TCP 80	0.0.0.0/0	Salidas HTTP autorizadas
nacl-web	OUT	115	TCP 443	0.0.0.0/0	Salida HTTPS y NAT público
nacl-web	OUT	120	TCP 8080	10.0.4.0/23	Web hacia aplicación
nacl-web	OUT	130	TCP 22	10.0.4.0/23	Bastión hacia aplicación
nacl-web	OUT	140	TCP 22	10.0.6.0/25	Bastión hacia datos
nacl-app	IN	100	TCP 8080	10.0.0.0/22	Solicitudes desde web
nacl-app	IN	110	TCP 22	10.0.0.0/22	SSH desde bastión en web
nacl-app	IN	120	TCP 1024-65535	0.0.0.0/0	Retorno hacia app para conexiones HTTPS y DB
nacl-app	OUT	100	TCP 1024-65535	10.0.0.0/22	Respuesta a web y bastión
nacl-app	OUT	110	TCP 5432	10.0.6.0/25	Consulta a datos
nacl-app	OUT	120	TCP 443	0.0.0.0/0	Actualizaciones vía NAT

NACL	Flujo	N.º	Puerto/protocolo	CIDR origen/destino	Motivo
nacl-datos	IN	100	TCP 5432	10.0.4.0/23	PostgreSQL desde app
nacl-datos	IN	110	TCP 22	10.0.0.0/22	SSH desde bastión
nacl-datos	OUT	100	TCP 1024-65535	10.0.4.0/23	Respuestas SQL a aplicación
nacl-datos	OUT	110	TCP 1024-65535	10.0.0.0/22	Respuestas SSH a bastión

Precisión: la regla de entrada efímera de la NACL de aplicación está abierta a 0.0.0.0/0 para respuestas procedentes de Internet por NAT y de distintas conexiones iniciadas por la app. No equivale a exposición pública de las instancias: no tienen IP públicas ni ruta al IGW, y los Security Groups permanecen restrictivos. Aun así, para un entorno real se aconseja separar la salida a Internet por subred y ajustar reglas/puertos según las necesidades observadas.

III. Arquitectura de seguridad del proyecto

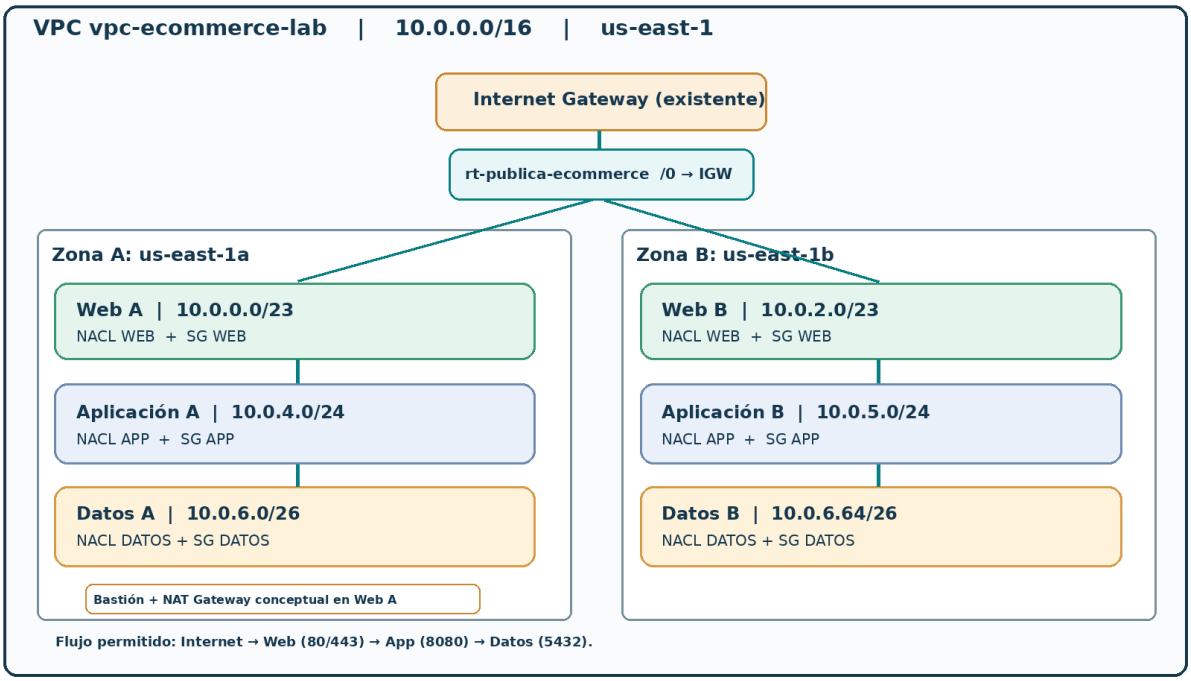


Figura 1. Arquitectura propia de Seguridad Perimetral sobre la VPC del Laboratorio 2. El NAT Gateway y el bastión son componentes propuestos para la Ruta B, no recursos desplegados.

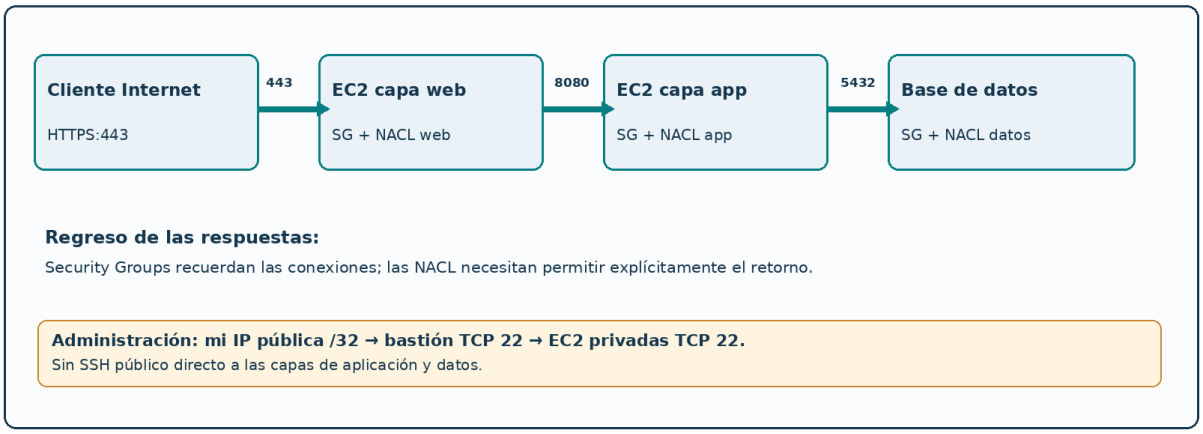


Figura 2. Flujos autorizados entre Internet, web, aplicación y datos y acceso administrativo mediante bastión. Elaboración propia.

3.1 Tablas de ruteo después de incorporar NAT (diseño propuesto)

Tabla	Subredes	Destino	Target propuesto
rt-publica-ecommerce	Web A y Web B	10.0.0.0/16	local
rt-publica-ecommerce	Web A y Web B	0.0.0.0/0	igw-ecommerce
rt-privada-ecommerce	App A/B + Datos A/B	10.0.0.0/16	local
rt-privada-ecommerce	App A/B + Datos A/B	0.0.0.0/0	nat-ecommerce-zona-a (solo diseño)

Con la tabla privada compartida se simplifica el Laboratorio 3, pero la ruta /0 pasa a estar disponible también para datos. La salida seguirá condicionada por los SG y NACL de cada servidor. Una mejora de mínimo privilegio consistiría en una tabla sin /0 para datos y otra con /0 hacia NAT para aplicación. Para resiliencia multi-AZ convendría además un NAT zonal en B y tablas de ruta privadas por zona. Esa variante no se confunde con la topología básica diseñada en el Lab 2. [6, 9]

3.2 NAT Gateway no equivale a una instancia EC2

La Ruta B describe un NAT Gateway gestionado por AWS en la subred pública sn-web-zona-a. Este recurso no lleva asociado Security Group: las reglas del SG de una NAT Instance no se trasladan al gateway administrado. Las NACL de su subred sí se aplican a su tránsito, además de las rutas. AWS cobra el NAT Gateway por tiempo aprovisionado y datos procesados, de modo que no lo crearía en una práctica documental. [6, 9, 13]

IV. Manual de configuración documentada en AWS

4.1 Preparación y comprobación de la infraestructura heredada

1. Seleccionar región. Abrir la consola de AWS y seleccionar us-east-1. Todos los componentes deben consultarse en la misma región.

2. Buscar la VPC. Ingresar a VPC → Your VPCs, localizar la etiqueta vpc-ecommerce-lab y verificar el CIDR 10.0.0.0/16. En una ejecución se registraría su VPC ID real.

3. Revisar las seis subredes. En VPC → Subnets comparar nombre, CIDR y AZ con el inventario del Lab 2. Las subredes web serán las únicas públicas.

4. Comprobar tablas. En VPC → Route tables confirmar rt-publica-ecommerce y rt-privada-ecommerce, con sus asociaciones 2/4. No crear una segunda VPC ni recalcular CIDR.

5. Comprobar IGW. En VPC → Internet gateways comprobar que igw-ecommerce está Attached a la VPC. Si no existiera, se documentaría primero su creación y asociación, pero no se duplica innecesariamente.

4.2 Crear Security Groups y aplicar mínimo privilegio

1. Entrar a Security Groups. Ir a EC2 → Network & Security → Security Groups → Create security group. Alternativamente abrir VPC → Security groups. [11]

2. Seleccionar VPC y nombres válidos. Elegir vpc-ecommerce-lab. Usar como nombre interno ecommerce-web y etiqueta Name=sg-web. Repetir ecommerce-app/sg-app, ecommerce-datos/sg-datos y ecommerce-bastion/sg-bastion. No utilizar sg- como prefijo de GroupName. [1]

3. Diseñar las reglas antes de guardarlas. Para sg-web permitir entradas HTTP/HTTPS de 0.0.0.0/0 y SSH desde sg-bastion; sg-app desde sg-web:8080 y sg-bastion:22; sg-datos desde sg-app:5432 y sg-bastion:22; sg-bastion SSH solo desde [MI_IP]/32.

4. Restringir salidas. Eliminar Allow all outbound de cada grupo y agregar los destinos de la matriz: web→app:8080, app→datos:5432, bastión→servidores:22 y HTTPS si está justificado.

5. Asociar a recursos futuros. Al lanzar o editar las instancias, seleccionar su SG correspondiente. Hasta asociarlo a una ENI, la existencia del SG no demuestra que esté protegiendo un servidor.

6. Comprobar. Abrir las pestañas Inbound rules y Outbound rules y comparar protocolos, puertos y fuentes/destinos con la matriz. Resultado esperado: no hay permisos directos 0.0.0.0/0 para sg-app ni sg-datos.

Nota técnica. Los Security Groups creados sin entradas y con salida permitida por defecto deben configurarse antes de usarlos. Las reglas SG son con estado y no requieren abrir explícitamente los puertos efímeros de respuesta. [1, 2]

4.3 Crear y asociar NACLs por capa

1. Crear ACL personalizada. En VPC → Network ACLs → Create network ACL, seleccionar la VPC, guardar las etiquetas nacl-web, nacl-app y nacl-datos. [12]

2. Recordar el bloqueo inicial. Una NACL nueva trae DENY * implícito en ambos sentidos; no asociarla todavía a subredes activas hasta preparar reglas completas.

3. Configurar nacl-web. En Inbound rules crear 80/443 desde Internet, 22 desde [MI_IP]/32 y retorno efímero. En Outbound rules permitir respuestas efímeras, salidas HTTPS/HTTP y tráfico interno a aplicación/datos para el bastión según la tabla 2.4.

4. Configurar nacl-app. Permitir inbound 8080 desde CIDR web, 22 desde CIDR web/bastión y retornos efímeros. En outbound permitir respuestas hacia web, solicitudes a datos:5432 y conexiones HTTPS que saldrían por NAT.

5. Configurar nacl-datos. Permitir inbound 5432 solo desde CIDR aplicación y 22 desde CIDR web/bastión. En outbound permitir respuestas efímeras correspondientes; no permitir salida abierta a Internet.

6. Definir números y decisiones. Usar secuencia 100,110,120, etc. Cada regla tendrá ALLOW o DENY, dirección Inbound/Outbound, protocolo, puerto y CIDR. Las reglas no incluidas terminan en DENY *.

7. Asociar subredes. En Subnet associations, asignar las dos web a nacl-web, las dos app a nacl-app y ambas datos a nacl-datos. Se reemplaza la asociación anterior; una subred tiene una NACL efectiva a la vez.

8. Verificar. Revisar las seis asociaciones y confirmar que no haya errores en el retorno efímero. Un filtro sin estado necesita reglas simétricas de tránsito, aunque no sean puertos idénticos.

4.4 Verificar Internet Gateway e investigar un NAT Gateway público

1. Verificar el Internet Gateway existente. VPC → Internet gateways. Confirmar igw-ecommerce Attached a vpc-ecommerce-lab y 0.0.0.0/0→igw en rt-publica-ecommerce. [5]

2. Abrir NAT gateways. VPC → NAT gateways → Create NAT gateway. Elegir nombre nat-ecommerce-zona-a y subnet sn-web-zona-a, que es pública. [7]

3. Elegir conectividad. Seleccionar Public y el modo zonal si se muestra la opción, conservando el diseño por AZ. Asignar/seleccionar una Elastic IP compatible con la zona. No se debe seleccionar Private NAT gateway para salida al Internet público.

4. Crear y esperar disponibilidad (hipotético). Al confirmar, NAT gateway pasa de Pending a Available. La Ruta B documenta ese comportamiento pero no crea el recurso porque generaría cargos. [6, 7]

5. Modificar tabla privada (hipotético). En rt-privada-ecommerce → Routes → Edit routes → Add route, escribir destino 0.0.0.0/0 y target nat-ecommerce-zona-a. No modificar la ruta local ni apuntar las privadas al IGW.

6. Verificar retorno y coherencia. Comprobar que el NAT Gateway tenga estado Available, se encuentre en subred pública y que la tabla privada resuelva /0 a NAT. Un servidor privado solo podrá acceder a Internet si sus SG y NACL permiten el tráfico.

7. Mejora por zona. Para alta disponibilidad real se propondría nat-ecommerce-zona-b en Web B y rutas privadas separadas por AZ. Como no son exigidos aquí recursos reales, se presenta como recomendación de diseño, no como ejecución.

4.5 Diseñar y documentar el bastión administrativo

1. Crear Security Group. El grupo identificado con Name=sg-bastion tendrá inbound SSH TCP 22 desde [MI_IP]/32 y outbound SSH 22 hacia los grupos de servidores internos. Nunca usar 0.0.0.0/0 para su SSH.

2. Planificar instancia. EC2 → Instances → Launch instances. Elegir Amazon Linux 2023, un tipo pequeño compatible, un par de claves autorizado y nombre ec2-bastion. No adjudicar un AMI ID ni IP pública ficticios.

3. Elegir la red. Seleccionar vpc-ecommerce-lab y sn-web-zona-a; habilitar asignación de IP pública solo para el bastión que se conectaría desde fuera. Asociar el grupo sg-bastion.

4. Preparar instancias privadas. App y Datos permanecerían sin IP pública, con las entradas SSH permitidas solo desde sg-bastion y con sus NACL permitiendo ambos sentidos.

5. Verificar acceso controlado (hipotético). Con el bastión disponible, se conectaría desde la IP administrativa mediante SSH y desde esa sesión a las IP privadas de la aplicación. Un SSH directo por Internet a una instancia sin IP pública debe no funcionar; no se simulan capturas de terminal.

6. Considerar alternativa administrada. AWS documenta Systems Manager Session Manager y EC2 Instance Connect como alternativa para evitar puertos SSH entrantes abiertos. Se menciona como mejora futura, no como parte implementada de este laboratorio. [8]

V. Comandos AWS CLI equivalentes

Los ejemplos siguientes son para Bash con AWS CLI v2 y credenciales configuradas en un contexto autorizado. Son una receta documental: no ejecutar sin comprender los cargos y permisos asociados. Se reutilizan los identificadores que la API devolvería, nunca se inventan valores. Las variables del Lab 2, como VPC_ID, WEB_A y APP_A, se obtendrían de una cuenta real. [14–20]

5.1 Variables y recuperación de recursos del Lab 2

```
export AWS_DEFAULT_REGION=us-east-1
export AWS_PAGER=""
# Consultas (no crean recursos):
aws ec2 describe-vpcs --filters Name=tag:Name,Values=vpc-ecommerce-lab
aws ec2 describe-subnets --filters Name=tag:Name,Values=sn-web-zona-a
# Confirmar IDs exactos antes de definir las variables.
VPC_ID="<vpc-id-real>"
WEB_A="<subnet-id-web-a-real>"
WEB_B="<subnet-id-web-b-real>"
APP_A="<subnet-id-app-a-real>"
APP_B="<subnet-id-app-b-real>"
DATOS_A="<subnet-id-datos-a-real>"
DATOS_B="<subnet-id-datos-b-real>"
RT_PRIVADA="<rtb-id-privada-real>"
ADMIN_CIDR="<IP-publica-real>/32"
```

Los campos entre signos angulares son marcadores que se reemplazarían con datos de una cuenta autorizada. Los comandos que cambian recursos no se han ejecutado. La etiqueta Name no es el ID y puede repetirse, por lo que conviene revisar antes de asignar variables.

5.2 Security Groups: crear y configurar

```
SG_WEB=$(aws ec2 create-security-group \
  --vpc-id "$VPC_ID" --group-name ecommerce-web \
  --description "Web ecommerce" --tag-specifications \
  'ResourceType=security-group,Tags=[{Key=Name,Value=sg-web}]' \
  --query GroupId --output text)
SG_APP=$(aws ec2 create-security-group \
  --vpc-id "$VPC_ID" --group-name ecommerce-app \
```

```
  --description "Aplicacion privada" \
  --query GroupId --output text)
SG_DATOS=$(aws ec2 create-security-group \
  --vpc-id "$VPC_ID" --group-name ecommerce-datos \
  --description "PostgreSQL privada" \
  --query GroupId --output text)
SG_BASTION=$(aws ec2 create-security-group \
  --vpc-id "$VPC_ID" --group-name ecommerce-bastion \
  --description "Acceso administrativo" \
  --query GroupId --output text)
# Etiquetas Name de los restantes:
aws ec2 create-tags --resources "$SG_APP" --tags Key=Name,Value=sg-app
aws ec2 create-tags --resources "$SG_DATOS" --tags Key=Name,Value=sg-datos
aws ec2 create-tags --resources "$SG_BASTION" --tags Key=Name,Value=sg-bastion
```

create-security-group recibe un nombre interno válido, la VPC y una descripción; --query GroupId toma el identificador real que se usaría como referencia. create-tags establece el nombre visible del diseño. [14]

```
# Reglas de entrada usando SG de origen:
aws ec2 authorize-security-group-ingress --group-id "$SG_WEB" \
  --protocol tcp --port 443 --cidr 0.0.0.0/0
aws ec2 authorize-security-group-ingress --group-id "$SG_WEB" \
  --protocol tcp --port 80 --cidr 0.0.0.0/0
aws ec2 authorize-security-group-ingress --group-id "$SG_WEB" \
  --protocol tcp --port 22 --source-group "$SG_BASTION"
aws ec2 authorize-security-group-ingress --group-id "$SG_APP" \
  --protocol tcp --port 8080 --source-group "$SG_WEB"
aws ec2 authorize-security-group-ingress --group-id "$SG_APP" \
  --protocol tcp --port 22 --source-group "$SG_BASTION"
aws ec2 authorize-security-group-ingress --group-id "$SG_DATOS" \
  --protocol tcp --port 5432 --source-group "$SG_APP"
aws ec2 authorize-security-group-ingress --group-id "$SG_DATOS" \
  --protocol tcp --port 22 --source-group "$SG_BASTION"
aws ec2 authorize-security-group-ingress --group-id "$SG_BASTION" \
```

```
  --protocol tcp --port 22 --cidr "$ADMIN_CIDR"
```

authorize-security-group-ingress añade permisos entrantes. --source-group identifica otro grupo como origen; --cidr acepta un rango IPv4. La etiqueta 0.0.0.0/0 solo aparece en HTTP/HTTPS de la capa pública. El marcador ADMIN_CIDR requiere la IP real del administrador. [15]

5.3 Restringir las reglas de salida que AWS agrega por defecto

```
# En un grupo nuevo puede existir Allow all IPv4 outbound:
for GROUP in "$SG_WEB" "$SG_APP" "$SG_DATOS" "$SG_BASTION"; do
  aws ec2 revoke-security-group-egress --group-id "$GROUP" \
    --ip-permissions 'IpProtocol=-1,IpRanges=[{CidrIp=0.0.0.0/0}]'
done
# Salidas iniciadas entre capas:
aws ec2 authorize-security-group-egress --group-id "$SG_WEB" \
  --protocol tcp --port 8080 --source-group "$SG_APP"
aws ec2 authorize-security-group-egress --group-id "$SG_APP" \
  --protocol tcp --port 5432 --source-group "$SG_DATOS"
aws ec2 authorize-security-group-egress --group-id "$SG_APP" \
  --protocol tcp --port 443 --cidr 0.0.0.0/0
aws ec2 authorize-security-group-egress --group-id "$SG_WEB" \
  --protocol tcp --port 443 --cidr 0.0.0.0/0
aws ec2 authorize-security-group-egress --group-id "$SG_BASTION" \
  --protocol tcp --port 22 --source-group "$SG_APP"
aws ec2 authorize-security-group-egress --group-id "$SG_BASTION" \
  --protocol tcp --port 22 --source-group "$SG_DATOS"
aws ec2 authorize-security-group-egress --group-id "$SG_BASTION" \
  --protocol tcp --port 22 --source-group "$SG_WEB"
aws ec2 authorize-security-group-egress --group-id "$SG_BASTION" \
  --protocol tcp --port 443 --cidr 0.0.0.0/0
```

revoke-security-group-egress elimina una salida permisiva si existe; hay que inspeccionar primero los permisos efectivos antes de intentar revocar. authorize-security-group-egress permite destinos específicos. sg-datos permanece sin reglas de salida iniciada; las respuestas a entradas autorizadas se aceptan por el estado del SG. [16]

5.4 NACL: creación, reglas y asociación

```
ACL_WEB=$(aws ec2 create-network-acl --vpc-id "$VPC_ID" \
  --query 'NetworkAcl.NetworkAclId' --output text)
ACL_APP=$(aws ec2 create-network-acl --vpc-id "$VPC_ID" \
  --query 'NetworkAcl.NetworkAclId' --output text)
ACL_DATOS=$(aws ec2 create-network-acl --vpc-id "$VPC_ID" \
  --query 'NetworkAcl.NetworkAclId' --output text)
aws ec2 create-tags --resources "$ACL_WEB" --tags Key=Name,Value=nacl-web
aws ec2 create-tags --resources "$ACL_APP" --tags Key=Name,Value=nacl-app
aws ec2 create-tags --resources "$ACL_DATOS" --tags Key=Name,Value=nacl-datos
```

Cada create-network-acl crea una NACL en la VPC; inicialmente no tiene reglas Allow, de modo que se configuran por completo antes de asociarla a subredes. [17]

```
# Ejemplo mínimo de nacl-web; agregar TODAS las reglas de tabla 2.4:
aws ec2 create-network-acl-entry --network-acl-id "$ACL_WEB" \
  --ingress --rule-number 100 --protocol tcp \
  --port-range From=443,To=443 --cidr-block 0.0.0.0/0 \
  --rule-action allow
aws ec2 create-network-acl-entry --network-acl-id "$ACL_WEB" \
  --ingress --rule-number 110 --protocol tcp \
  --port-range From=22,To=22 --cidr-block "$ADMIN_CIDR" \
  --rule-action allow
aws ec2 create-network-acl-entry --network-acl-id "$ACL_WEB" \
  --egress --rule-number 100 --protocol tcp \
  --port-range From=1024,To=65535 --cidr-block 0.0.0.0/0 \
  --rule-action allow
# Ejemplo para tráfico app a datos:
aws ec2 create-network-acl-entry --network-acl-id "$ACL_DATOS" \
  --ingress --rule-number 100 --protocol tcp \
  --port-range From=5432,To=5432 \
  --cidr-block 10.0.4.0/23 --rule-action allow
aws ec2 create-network-acl-entry --network-acl-id "$ACL_DATOS" \
  --egress --rule-number 100 --protocol tcp \
  --port-range From=1024,To=65535 \
```

```
  --cidr-block 10.0.4.0/23 --rule-action allow
```

Las siguientes reglas completan el conjunto propuesto en la matriz 2.4. Deben revisarse antes de aplicarse, junto con las reglas de la interfaz del sistema y el modelo de tráfico. Una NACL recién creada permanece sin asociaciones hasta configurar todas sus entradas. [18]

5.4.1 Comandos completos de la matriz NACL

```
# nacl-web IN | HTTP para redirección
aws ec2 create-network-acl-entry --network-acl-id "$ACL_WEB" --ingress \
  --rule-number 100 --protocol tcp --port-range From=80,To=80 \
  --cidr-block 0.0.0.0/0 --rule-action allow
# nacl-web IN | HTTPS público
aws ec2 create-network-acl-entry --network-acl-id "$ACL_WEB" --ingress \
  --rule-number 105 --protocol tcp --port-range From=443,To=443 \
  --cidr-block 0.0.0.0/0 --rule-action allow
# nacl-web IN | SSH al bastión
aws ec2 create-network-acl-entry --network-acl-id "$ACL_WEB" --ingress \
  --rule-number 110 --protocol tcp --port-range From=22,To=22 \
  --cidr-block "$ADMIN_CIDR" --rule-action allow
# nacl-web IN | Retorno de conexiones salientes del bastión y NAT
aws ec2 create-network-acl-entry --network-acl-id "$ACL_WEB" --ingress \
  --rule-number 120 --protocol tcp --port-range From=1024,To=65535 \
  --cidr-block 0.0.0.0/0 --rule-action allow
# nacl-web OUT | Respuestas de web y SSH
aws ec2 create-network-acl-entry --network-acl-id "$ACL_WEB" --egress \
  --rule-number 100 --protocol tcp --port-range From=1024,To=65535 \
  --cidr-block 0.0.0.0/0 --rule-action allow
# nacl-web OUT | Salidas HTTP autorizadas
aws ec2 create-network-acl-entry --network-acl-id "$ACL_WEB" --egress \
  --rule-number 110 --protocol tcp --port-range From=80,To=80 \
  --cidr-block 0.0.0.0/0 --rule-action allow
# nacl-web OUT | Salida HTTPS y NAT público
aws ec2 create-network-acl-entry --network-acl-id "$ACL_WEB" --egress \
  --rule-number 115 --protocol tcp --port-range From=443,To=443 \
  --cidr-block 0.0.0.0/0 --rule-action allow
```

```
# nacl-web OUT | Web hacia aplicación
aws ec2 create-network-acl-entry --network-acl-id "$ACL_WEB" --egress \
  --rule-number 120 --protocol tcp --port-range From=8080,To=8080 \
  --cidr-block 10.0.4.0/23 --rule-action allow
# nacl-web OUT | Bastión hacia aplicación
aws ec2 create-network-acl-entry --network-acl-id "$ACL_WEB" --egress \
  --rule-number 130 --protocol tcp --port-range From=22,To=22 \
  --cidr-block 10.0.4.0/23 --rule-action allow
# nacl-web OUT | Bastión hacia datos
aws ec2 create-network-acl-entry --network-acl-id "$ACL_WEB" --egress \
  --rule-number 140 --protocol tcp --port-range From=22,To=22 \
  --cidr-block 10.0.6.0/25 --rule-action allow
# nacl-app IN | Solicitudes desde web
aws ec2 create-network-acl-entry --network-acl-id "$ACL_APP" --ingress \
  --rule-number 100 --protocol tcp --port-range From=8080,To=8080 \
  --cidr-block 10.0.0.0/22 --rule-action allow
# nacl-app IN | SSH desde bastión en web
aws ec2 create-network-acl-entry --network-acl-id "$ACL_APP" --ingress \
  --rule-number 110 --protocol tcp --port-range From=22,To=22 \
  --cidr-block 10.0.0.0/22 --rule-action allow
# nacl-app IN | Retorno hacia app para conexiones HTTPS y DB
aws ec2 create-network-acl-entry --network-acl-id "$ACL_APP" --ingress \
  --rule-number 120 --protocol tcp --port-range From=1024,To=65535 \
  --cidr-block 0.0.0.0/0 --rule-action allow
# nacl-app OUT | Respuesta a web y bastión
aws ec2 create-network-acl-entry --network-acl-id "$ACL_APP" --egress \
  --rule-number 100 --protocol tcp --port-range From=1024,To=65535 \
  --cidr-block 10.0.0.0/22 --rule-action allow
# nacl-app OUT | Consulta a datos
aws ec2 create-network-acl-entry --network-acl-id "$ACL_APP" --egress \
  --rule-number 110 --protocol tcp --port-range From=5432,To=5432 \
  --cidr-block 10.0.6.0/25 --rule-action allow
# nacl-app OUT | Actualizaciones vía NAT
aws ec2 create-network-acl-entry --network-acl-id "$ACL_APP" --egress \
```

```
  --rule-number 120 --protocol tcp --port-range From=443,To=443 \
  --cidr-block 0.0.0.0/0 --rule-action allow
# nacl-datos IN | PostgreSQL desde app
aws ec2 create-network-acl-entry --network-acl-id "$ACL_DATOS" --ingress \
  --rule-number 100 --protocol tcp --port-range From=5432,To=5432 \
  --cidr-block 10.0.4.0/23 --rule-action allow
# nacl-datos IN | SSH desde bastión
aws ec2 create-network-acl-entry --network-acl-id "$ACL_DATOS" --ingress \
  --rule-number 110 --protocol tcp --port-range From=22,To=22 \
  --cidr-block 10.0.0.0/22 --rule-action allow
# nacl-datos OUT | Respuestas SQL a aplicación
aws ec2 create-network-acl-entry --network-acl-id "$ACL_DATOS" --egress \
  --rule-number 100 --protocol tcp --port-range From=1024,To=65535 \
  --cidr-block 10.0.4.0/23 --rule-action allow
# nacl-datos OUT | Respuestas SSH a bastión
aws ec2 create-network-acl-entry --network-acl-id "$ACL_DATOS" --egress \
  --rule-number 110 --protocol tcp --port-range From=1024,To=65535 \
  --cidr-block 10.0.0.0/22 --rule-action allow
```

No se asignan reglas sueltas a recursos reales en esta ruta. Si se emplearan estas órdenes, se verificaría antes que cada ID esté definido, que los rangos de puertos sean válidos y que las reglas no se dupliquen. La última regla * continúa denegando los demás paquetes. [18]

```
# Buscar el ID ACTUAL de asociación de sn-app-zona-a:
aws ec2 describe-network-acls \
  --filters "Name=association.subnet-id,Values=$APP_A" \
  --query 'NetworkAcls[].Associations[]' --output table
# Copiar el NetworkAclAssociationId real de esa respuesta:
ACL_ASSOC_APP_A="<aclassoc-id-real>"
aws ec2 replace-network-acl-association \
  --association-id "$ACL_ASSOC_APP_A" \
  --network-acl-id "$ACL_APP"
```

replace-network-acl-association necesita el ID de asociación vigente, que no es igual al ID de subred ni al de NACL. Se repetiría con las restantes cinco subredes, asignando web a ACL_WEB, app a ACL_APP y datos a ACL_DATOS. Una asociación válida no prueba por sí sola que las reglas dejan pasar las aplicaciones. [19]

5.5 Gateway, rutas y bastión: comandos de referencia

El Laboratorio 2 ya dejó diseñado el Internet Gateway. Por eso lo correcto sería comprobar su existencia y no crear un segundo. No obstante, el equivalente de la acción de consola es:

```
# Solo si el IGW no existe aún; no duplicar recursos
IGW_ID=$(aws ec2 create-internet-gateway   --query InternetGateway.InternetGatewayId --output text)
aws ec2 attach-internet-gateway   --internet-gateway-id "$IGW_ID" --vpc-id "$VPC_ID"
# Antes de modificar rutas, describir los gateways asociados.
aws ec2 describe-internet-gateways   --filters "Name=attachment.vpc-id,Values=$VPC_ID"


# Verificación del IGW heredado:
aws ec2 describe-internet-gateways \
  --filters "Name=attachment.vpc-id,Values=$VPC_ID"
# Solo en una ejecución consciente del costo:
EIP_ALLOC=$(aws ec2 allocate-address --domain vpc \
  --query AllocationId --output text)
NAT_ID=$(aws ec2 create-nat-gateway \
  --subnet-id "$WEB_A" --connectivity-type public \
  --availability-mode zonal --allocation-id "$EIP_ALLOC" \
  --query 'NatGateway.NatGatewayId' --output text)
aws ec2 wait nat-gateway-available --nat-gateway-ids "$NAT_ID"
aws ec2 create-route --route-table-id "$RT_PRIVADA" \
  --destination-cidr-block 0.0.0.0/0 --nat-gateway-id "$NAT_ID"
```

allocate-address reserva una Elastic IP; create-nat-gateway define subred pública y EIP, además del modo zonal en CLI vigente; create-route registra el NAT como target para /0 en la tabla privada. AWS puede facturar recursos desde su aprovisionamiento. Aquí los comandos son equivalencia investigada: no se ejecutan. En una tabla que ya tiene /0, se inspeccionaría y corregiría la ruta en lugar de crear una duplicada. [20, 21]

```
# Lanzamiento ilustrativo de bastión: reemplazar AMI y llave con datos válidos
AMI_ID="<ami-id-real-de-Amazon-Linux-2023>"
KEY_NAME="<par-de-claves-existente>"
aws ec2 run-instances --image-id "$AMI_ID" \
  --instance-type t3.micro --count 1 --key-name "$KEY_NAME" \
  --network-interfaces
"DeviceIndex=0,SubnetId=$WEB_A,Groups=$SG_BASTION,AssociatePublicIpAddress=true"
\
  --tag-specifications 'ResourceType=instance,Tags=[{Key=Name,Value=ec2-bastion}]'
```

run-instances ilustra la creación de un bastión y su interfaz en una subred pública. El tipo de instancia, AMI, clave SSH y beneficios gratuitos dependen de la cuenta. En Ruta B el objetivo es documentar el equivalente, sin desplegarla. No se solicita acceso privado directo por Internet. [22]

5.6 Significado de los parámetros principales

Parámetro	Qué representa en el proyecto
--vpc-id	ID real de VPC; no equivale a la etiqueta Name.
--group-name	Nombre interno válido del SG; se evita el prefijo reservado sg-.
--description	Texto descriptivo obligatorio al crear SG.
--group-id / --source-group	Identificador de SG receptor y referencia al grupo remoto como fuente/destino.
--protocol / --port	Protocolo TCP y puerto exacto autorizado.
--cidr / --cidr-block	Bloque de direcciones origen/destino, por ejemplo IP administrativa /32.
--ingress / --egress	Sentido de la regla NACL o SG.
--rule-number	Prioridad de NACL evaluada de menor a mayor.
--port-range From,To	Puertos de destino que coinciden con una regla NACL.
--rule-action allow/deny	Acción de una regla NACL; los SG no utilizan DENY.
--association-id	ID actual de asociación NACL–subred que reemplazará AWS.
--network-acl-id	NACL propuesta para una subred.

Parámetro	Qué representa en el proyecto
--allocation-id	Identificador de la Elastic IP de NAT Gateway público.
--subnet-id	Subred pública donde se ubicará NAT Gateway o bastión.
--availability-mode zonal	Modalidad de NAT Gateway por zona en CLI vigente.
--nat-gateway-id	Destino de la ruta privada por defecto.
--query / --output	Seleccionar campos de respuesta y presentación; no crea permisos.

VI. Verificación esperada, diagnóstico y reflexión

6.1 Lista de comprobación si se implementara

Objeto	Resultado previsto	Cómo revisarlo
VPC y subredes	Seis subredes originales sin cambios	VPC → Subnets
Security Groups	Web público solo 80/443; app y datos sin origen Internet	EC2 → Security Groups → Rules
NACL	Tres NACL, reglas numeradas completas y asociaciones 2+2+2	VPC → Network ACLs
Bastión	SSH desde una sola IP; salto privado a app/datos	Security Group y prueba autorizada
Internet Gateway	Attached a VPC; ruta solo en tabla pública	VPC → Internet gateways / Route tables
NAT Gateway	Public, Available, en Web A, EIP asociada	VPC → NAT gateways
Tabla privada	/0 → NAT y ruta local conservada (propuesta)	VPC → Route tables
Aislamiento	Datos sin IP pública, ingreso SQL solo desde app	Direcciones y SG

6.2 Errores comunes y método para resolverlos

Síntoma	Causa probable	Comprobación y corrección
DuplicatePermission	Regla SG repetida	Consultar reglas efectivas antes de autorizar otra.
InvalidGroup.NotFound	SG ID incorrecto, región o VPC diferente	Verificar describe-security-groups e ID.
GroupName con sg-	Prefijo reservado por AWS	Utilizar GroupName=ecommerce-web; Name tag=sg-web.
SSH a bastión falla	Mi IP cambió, IP pública ausente, regla SSH o NACL	Revisar IP/32, SG, NACL pública y estado EC2.
App 8080 no responde	SG de origen mal referenciado o NACL sin respuesta efímera	Verificar regla app ingress 8080 y retorno app→web.
DB no recibe SQL	Puerto distinto, regla sg-app o NACL faltante	Comparar 5432 entre app y datos; no abrir 0.0.0.0/0.

Síntoma	Causa probable	Comprobación y corrección
NAT Pending / Failed	EIP, cuota o AZ incompatible	Revisar estado y causa en NAT gateways.
App sin Internet	Falta ruta /0 a NAT, NACL o SG impide salida/retorno	Verificar /0, NAT Available, HTTPS e inbound efímero.
No se puede asociar NACL	AssociationId antiguo o NACL de otra VPC	Consultar asociación actual y repetir reemplazo.
Rutas duplicadas	Existe /0 anterior	Describe route tables; no crear duplicada.
NACL bloquea todo	Nueva ACL con DENY * sin ALLOW	Completar reglas antes de asociar.

El método sería diagnosticar desde las capas: primero IP, subred y tabla efectiva; después Security Group; luego NACL y puertos de retorno; finalmente estado de la instancia y servicio. Una conexión fallida no demuestra por sí sola que se rechazó por el firewall, porque también podría faltar ruta o proceso de destino. [3, 12, 23]

6.3 Reflexión final sobre resultados esperados y errores

Si implementara este diseño en mi cuenta AWS, no daría la práctica por terminada solo por ver cuatro Security Groups y tres NACL con nombres correctos. Empezaría comprobando que todos pertenecen a la VPC 10.0.0.0/16 y que las seis subredes siguen distribuidas como en el Laboratorio 2. En cada SG revisaría tanto las reglas entrantes como las salientes, especialmente la ausencia de 0.0.0.0/0 hacia aplicación y base de datos. Después verificaría que las NACL tengan las asociaciones correctas y que el retorno por puertos efímeros se encuentre permitido en ambas direcciones. Esto último es importante porque un error de salida puede hacer fallar una conexión que parece estar autorizada por el SG.

Para el NAT Gateway esperaría observar estado Available, una Elastic IP asociada y una ruta 0.0.0.0/0 desde la tabla privada hacia su ID, nunca directamente hacia el Internet Gateway. En el caso del bastión, la conexión administrativa tendría que entrar únicamente desde mi IP pública y continuar hacia las direcciones privadas. Revisaría el intento de acceso directo desde Internet como una prueba negativa; tampoco afirmaría aislamiento solo porque falle un ping. Ante errores como permisos duplicados, ACL que bloquean el retorno o NAT no disponible, inspeccionaría primero las dependencias antes de repetir comandos. Lo principal que aprendí es que la seguridad de una red en AWS surge de varias decisiones coherentes y no de una única casilla marcada. Esta entrega sigue siendo una investigación documentada, no una validación práctica ejecutada.

VII. Evidencias oficiales y control de rúbrica

La guía de Ruta B solicita al menos 2 a 3 capturas auténticas de la documentación oficial de AWS o de un tutorial/demo oficial, citando la URL y la fecha de consulta. En esta versión se incorporan dos evidencias oficiales diferentes, suficientes para cubrir el requisito mínimo del laboratorio. El tercer archivo recibido repite visualmente la misma página de Network ACLs, por lo que se conserva un espacio opcional por si luego se desea reemplazarlo con una captura distinta, por ejemplo de Security Groups. [24]

E1. Network ACLs — creación y reglas

Captura oficial de la página “Create a network ACL for your VPC”, donde se observa el título del procedimiento y los pasos de creación, reglas y asociación.



Figura E1. Evidencia oficial de AWS sobre creación de Network ACLs.

Fuente: https://docs.aws.amazon.com/vpc/latest/userguide/create-network-acl.html

E2. NAT Gateway — creación y requisitos

Captura oficial de la página “Work with NAT gateways”, donde se observan las tareas principales y el apartado “Create a NAT gateway”.

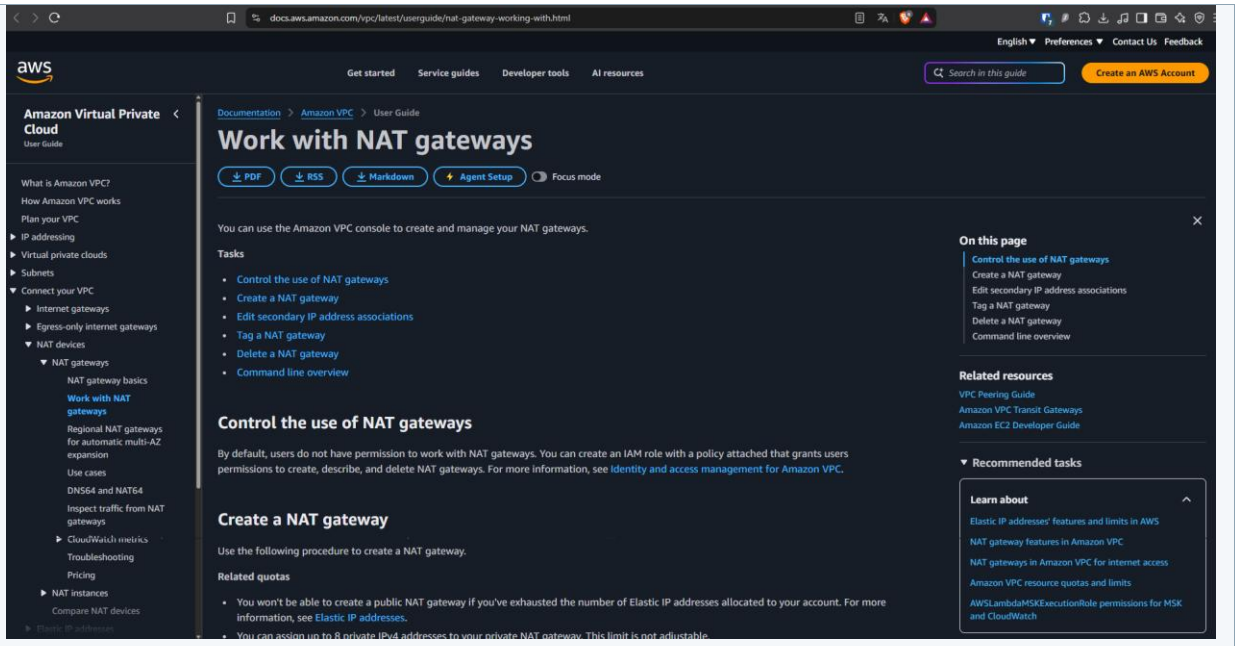


Figura E2. Evidencia oficial de AWS sobre trabajo con NAT Gateways.

Fuente: *https://docs.aws.amazon.com/vpc/latest/userguide/nat-gateway-working-with.html*

VIII. Referencias oficiales

Fuentes técnicas oficiales de Amazon Web Services, consultadas para el desarrollo de la investigación el 11 de octubre de 2026. Los procedimientos son documentación equivalente a Ruta A, no resultados obtenidos desde una cuenta. Los parámetros de la documentación pueden actualizarse posteriormente.

[1] AWS. Security groups for your VPC. https://docs.aws.amazon.com/vpc/latest/userguide/vpc-security-groups.html

[2] AWS. Security group rules. https://docs.aws.amazon.com/vpc/latest/userguide/security-group-rules.html

[3] AWS. Network ACL basics. https://docs.aws.amazon.com/vpc/latest/userguide/vpc-network-acls.html

[4] AWS. Custom network ACLs (ephemeral ports). https://docs.aws.amazon.com/vpc/latest/userguide/custom-network-acl.html

[5] AWS. Internet gateways. https://docs.aws.amazon.com/vpc/latest/userguide/VPC_Internet_Gateway.html

[6] AWS. NAT gateways. https://docs.aws.amazon.com/vpc/latest/userguide/vpc-nat-gateway.html

[7] AWS. Work with NAT gateways. https://docs.aws.amazon.com/vpc/latest/userguide/nat-gateway-working-with.html

[8] AWS. Bastion with Session Manager and Instance Connect. https://docs.aws.amazon.com/prescriptive-guidance/latest/patterns/access-a-bastion-host-by-using-session-manager-and-amazon-ec2-instance-connect.html

[9] AWS. Compare NAT gateway and NAT instance. https://docs.aws.amazon.com/vpc/latest/userguide/vpc-nat-comparison.html

[10] AWS. Regional NAT gateways. https://docs.aws.amazon.com/vpc/latest/userguide/nat-gateways-regional.html

[11] AWS. Create a security group. https://docs.aws.amazon.com/vpc/latest/userguide/creating-security-groups.html

[12] AWS. Create a network ACL. https://docs.aws.amazon.com/vpc/latest/userguide/create-network-acl.html

[13] AWS. Amazon VPC pricing. https://aws.amazon.com/vpc/pricing/

[14] AWS. AWS CLI create-security-group. https://docs.aws.amazon.com/cli/latest/reference/ec2/create-security-group.html

[15] AWS. AWS CLI authorize-security-group-ingress.
https://docs.aws.amazon.com/cli/latest/reference/ec2/authorize-security-group-ingress.html

[16] AWS. AWS CLI authorize-security-group-egress.
https://docs.aws.amazon.com/cli/latest/reference/ec2/authorize-security-group-egress.html

[17] AWS. AWS CLI create-network-acl.
https://docs.aws.amazon.com/cli/latest/reference/ec2/create-network-acl.html

[18] AWS. AWS CLI create-network-acl-entry.
https://docs.aws.amazon.com/cli/latest/reference/ec2/create-network-acl-entry.html

[19] AWS. AWS CLI replace-network-acl-association.
https://docs.aws.amazon.com/cli/latest/reference/ec2/replace-network-acl-association.html

[20] AWS. AWS CLI create-nat-gateway.
https://docs.aws.amazon.com/cli/latest/reference/ec2/create-nat-gateway.html

[21] AWS. AWS CLI create-route.
https://docs.aws.amazon.com/cli/latest/reference/ec2/create-route.html

[22] AWS. AWS CLI run-instances. https://docs.aws.amazon.com/cli/latest/reference/ec2/run-instances.html

[23] AWS. Troubleshoot NAT gateways.
https://docs.aws.amazon.com/vpc/latest/userguide/nat-gateway-troubleshooting.html

[24] AWS. Guía de Laboratorio 3. Documento académico facilitado por el docente.